

StockLite Implementation Plan

[StockLite Implementation Plan](#)

[Phase 0 — Setup & Discovery](#)

[Phase 1 — Core Data Layer \(Shared Foundation\)](#)

[Phase 2 — Inventory View \(/ \)](#)

[Phase 3 — Stock In / Stock Out](#)

[Phase 4 — Warehouse Transfer](#)

[Phase 5 — Transaction History \(/history \)](#)

[Phase 6 — Debugging Pass](#)

[Phase 7 — Stretch Feature: Low-Stock Summary](#)

[Phase 8 — Final Verification & Report](#)

[Ordering Rationale \(Why the Phases Are Sequenced This Way\)](#)

StockLite Implementation Plan

This document expands the phase plan for completing and debugging the StockLite warehouse inventory application into a fully explained, phase-by-phase guide. Each phase states **what** to do, **why** it matters, **how** to approach it, and **what “done” looks like**, so it can be handed directly to a developer or coding agent.

Phase 0 — Setup & Discovery

Goal: Understand the existing codebase completely before writing a single line of logic.

Why this phase exists

The project brief is explicit that StockLite is an *existing* application with UI, types, and seed data already built — the task is to complete it, not replace it. Skipping discovery is the single biggest risk: it leads to duplicate logic, mismatched types, broken UI contracts, and wasted rework. Every later phase depends on an accurate mental model built here.

What to do

1. Run `npm install`, then `npm run dev`, and confirm the app boots without errors. Open every existing route (`/` , `/stock` , `/transfer` , `/history` , `/login`) to see current behavior.
2. Walk the repository structure: `app/` , `lib/` , `components/` , and any `api/` routes. Note how routing, layouts, and shared components are organized.
3. Read `lib/types.ts` in full. Record the exact shape of every relevant type: `Product` , `Warehouse` , `Transaction` (and its `type` enum — e.g. `"IN"` | `"OUT"` | `"TRANSFER"`), and any stock/inventory record type.
4. Read `lib/seed-data.ts` . Determine:
 - How stock is keyed — is it a flat array of `{ productId, warehouseId, quantity }` rows, or nested objects per warehouse?
 - Whether the data is exported as a mutable module-level array/object (meaning mutation-in-place works across API calls) or needs a wrapper.
5. Skim `lib/auth.ts` only enough to confirm it is a non-functional skeleton, per the README — do not build real authentication.
6. Search the entire repo for `TOD0` comments (`grep -r "TOD0"`), and compile a complete list with file and line number. This becomes the master checklist.
7. Inspect the existing `GET /api/items` and `GET /api/transactions` handlers to learn the exact response shape/contract the UI components already expect — new write endpoints must return compatible shapes.
8. For each page, identify the data-fetching pattern in use (client-side `fetch` in a `useEffect` , SWR/React Query, or a Server Component with revalidation). This determines how “instant UI update after a mutation” will be implemented later — via `router.refresh()` , cache revalidation, an SWR `mutate()` call, or local optimistic state.

Exit criteria

A written map of data flow (seed data → API → page) and a consolidated TODO checklist exists before any implementation begins. No code changes are made in this phase.

Phase 1 — Core Data Layer (Shared Foundation)

Goal: Build one centralized, reusable mutation and validation layer that every feature (Stock In, Stock Out, Transfer, Low-Stock Summary) will call into.

Why this phase exists

Several of the bugs the assignment explicitly calls out — incorrect stock totals, `<` used instead of `<=` for low-stock, broken transfers that update only one warehouse — are exactly the kind of bug that happens when stock-adjustment logic is duplicated across multiple routes/components instead of written once and reused. Centralizing this logic now prevents that entire class of bug and makes Phase 6 (debugging) mostly a formality rather than a rescue mission.

What to do

1. Create a single module, e.g. `lib/inventory-store.ts`, that wraps direct access to `seed-data.ts`. All reads and writes to stock and transactions should pass through this module rather than being scattered across API route files.
2. Implement small, pure, independently testable helper functions:
 - `getStock(productId, warehouseId): number` — returns current quantity, `0` if no record exists.
 - `isLowStock(stock: number, threshold: number): boolean` — returns `stock <= threshold`. This is the *single* source of truth for the low-stock rule, used by the inventory filter, the debugging checks, and the stretch-goal summary panel alike.
 - `applyStockChange(productId, warehouseId, delta: number): void` — adjusts stock by a signed delta (positive for stock-in/transfer-in, negative for stock-out/transfer-out), creating a new stock record if one doesn't exist for that product/warehouse pair.
 - `recordTransaction(entry): Transaction` — appends a correctly-typed transaction record and returns it.
3. Decide the transfer-linking scheme now, since Phase 4 depends on it — e.g. adding a `transferGroupId` (a shared UUID) to both the “OUT of source” and “IN to destination” transaction records so the history page can visually pair them.
4. Do not wire this module into any API route yet — this phase is purely about building and mentally verifying the foundation.

Exit criteria

The store module compiles cleanly, exposes the four helpers above (or equivalent), and is not yet imported by any route handler.

Phase 2 — Inventory View (/)

Goal: Make the main inventory grid fully correct and filterable.

Why this phase exists

This is the page users see first and the one every other page's data indirectly reflects. Getting the low-stock rule exactly right here (`<=`, not `<`) is called out repeatedly in the brief because it's an easy off-by-one mistake with real consequences — a product sitting exactly at its reorder threshold must be flagged.

What to do

1. Confirm the grid renders: product name, category, warehouse, current stock, and reorder threshold, using existing table/UI components — do not rebuild the table.
2. Wire the **category filter** using the distinct set of categories present in the seed data (via a dropdown/select already in the UI, if present).
3. Wire the **low-stock filter** using the `isLowStock` helper from Phase 1 — never inline a separate `<` comparison here.
4. Ensure category and low-stock filters **compose with AND logic** — selecting a category and toggling low-stock should narrow the results by both conditions simultaneously, not act as two independent, overriding filters.
5. Add or verify an **empty state**: when the combination of filters yields zero results, show a clear message (e.g. “No products match the selected filters”) rather than a blank or broken table.

Manual test matrix

- Category filter alone
- Low-stock filter alone
- Category + low-stock together
- No filters (baseline — full list)
- A combination that yields zero results (verify empty state)

Exit criteria

All items in §29 “Inventory” of the acceptance criteria are checked off, and stock exactly equal to threshold visibly appears under the low-stock filter.

Phase 3 — Stock In / Stock Out

Goal: Implement working, validated stock adjustment for a single warehouse, both in the UI (/stock) and the backing API.

Why this phase exists

This is the first place real data mutation happens, so it sets the pattern (validation → mutation → transaction log → UI refresh) that Transfer will reuse in Phase 4. Getting validation right here — and enforcing it **server-side**, not just in the form — closes the specific vulnerability called out in the brief: a user bypassing the UI and calling the API directly with a bad payload.

What to do

1. Extend the API layer with write endpoints for stock-in and stock-out (check whether the existing stub file already defines a route/shape that should be reused rather than duplicated).
2. Implement **server-side validation**, independent of whatever the frontend form already checks:
 - Reject non-numeric input, `NaN`, `undefined`, zero, and negative quantities.
 - For stock-out specifically, reject any quantity greater than the current stock at that product/warehouse — insufficient stock must never be allowed through.
3. On a valid request:
 - Call `applyStockChange` with a positive delta (stock-in) or negative delta (stock-out) from Phase 1.
 - Call `recordTransaction` with product, warehouse, type, quantity, and timestamp.
 - Return the updated item/stock record in the response.
4. On the frontend, after a successful response, update the UI immediately — using whichever pattern Phase 0 identified (local state update, `SWR mutate`, or `router.refresh()`) — so the user does not need to manually reload the page.
5. Surface validation errors returned by the API using the existing form’s error-display convention — don’t invent a new error UI pattern.

Edge case test matrix

Operation	Quantity	Expected result
Stock In	1	Accepted, stock +1
Stock In	0	Rejected
Stock In	-1	Rejected
Stock In	“abc”	Rejected
Stock In	very large valid number	Accepted
Stock Out	exactly equal to current stock	Accepted, stock → 0
Stock Out	less than current stock	Accepted
Stock Out	greater than current stock	Rejected, stock unchanged
Stock Out	0	Rejected
Stock Out	negative	Rejected
Stock Out	non-numeric	Rejected

Exit criteria

Every row above behaves as specified, both via the UI form and via a direct API call (e.g. curl/Postman) that bypasses the frontend entirely.

Phase 4 — Warehouse Transfer

Goal: Implement a transfer between two warehouses that is validated, atomic, and correctly recorded in history — the most failure-prone feature in the spec.

Why this phase exists

The brief singles this out as one of the most important requirements: a transfer must never leave inventory in a partially-updated state (source deducted but destination never credited). This phase reuses the Phase 1 helpers specifically so both sides of the transfer go through the same trusted mutation path.

What to do

1. Validate, **in this order, before any mutation happens**:
 - Product exists.
 - Source warehouse exists.
 - Destination warehouse exists.
 - Quantity is numeric and strictly greater than zero.
 - Quantity does not exceed current stock at the source.
2. Structure the transfer as a single function that:
 - Performs **all validation first**, completely, before calling any mutation function. This guarantees that if anything is invalid, nothing has been written yet — since the store is synchronous in-memory data, doing all checks before any write is what makes the operation effectively atomic (no partial state can occur because no write is attempted until every precondition is confirmed true).
 - Calls `applyStockChange(productId, sourceWarehouseId, -quantity)`.
 - Calls `applyStockChange(productId, destinationWarehouseId, +quantity)` — creating a new stock record at the destination if the product doesn't already exist there, or incrementing the existing one. Never discard the transferred quantity and never create a duplicate entry when one already exists.
 - Writes **two linked transaction records** — one representing the outbound movement from the source, one representing the inbound movement to the destination — sharing the `transferGroupId` (or equivalent field) decided in Phase 1, so the pair can be understood together in history.
3. On the frontend, after a successful transfer, refresh both the source and destination warehouse views so the change is visible without a manual reload.

Edge case test matrix

- Valid transfer with sufficient stock
- Transfer of exactly all available source stock (source should reach exactly 0)
- Transfer requesting more than available source stock (rejected, no mutation at all)
- Zero quantity (rejected)
- Negative quantity (rejected)
- Non-numeric quantity (rejected)
- Destination already holds the product (existing record incremented, no duplicate)
- Destination does not yet hold the product (new record created)
- Invalid source warehouse ID (rejected)
- Invalid destination warehouse ID (rejected)

Verification for atomicity specifically

After a rejected transfer (any invalid case above), explicitly confirm that **both** the source and destination stock values are byte-for-byte unchanged from before the attempt — not just that an error was shown.

Exit criteria

Every case above passes, total inventory across both warehouses is conserved after any successful transfer (source loss exactly equals destination gain), and no transaction is written unless the full transfer succeeded.

Phase 5 — Transaction History (/history)

Goal: Present a complete, filterable, correctly-sorted log of everything that happened to inventory.

Why this phase exists

History is the audit trail that lets a reviewer reconstruct what happened — it's only useful if it's complete, correctly typed, correctly filterable, and correctly ordered. This phase is largely a consumer of the transaction records written in Phases 3 and 4, so most of the real work here is filtering, sorting, and presentation.

What to do

1. Confirm `GET /api/transactions` returns everything the page needs: product, warehouse, transaction type, quantity, and timestamp, for every record — including both halves of transfer pairs.
2. Implement the **transaction type filter** (Stock In / Stock Out / Transfer, using the project's existing enum values — don't invent new ones).
3. Implement the **warehouse filter**.
4. Ensure both filters **compose with AND logic** (e.g. Type = Stock Out AND Warehouse = Warehouse A returns only records matching both).
5. Sort transactions by timestamp, **most recent first**. Use a non-mutating sort (`[...transactions].sort(...)`), since the underlying array/object may be shared with other parts of the app — sorting in place could silently corrupt data used elsewhere.
6. Make sure transfer pairs render in a way a reviewer can understand as one logical movement (e.g., grouped visually, or each row clearly labeled with direction and a shared reference/group ID).

Test matrix

- Type filter alone
- Warehouse filter alone
- Both filters together
- A combination with zero matching results (verify empty state)
- Sort order after a mix of stock-in, stock-out, and transfer operations performed in a non-chronological submission order

Exit criteria

All items in §29 “History” of the acceptance criteria are satisfied, and a reviewer can look at the list and clearly identify which product moved from which warehouse to which, and when.

Phase 6 — Debugging Pass

Goal: Explicitly re-verify the four named problem areas from the brief, as a dedicated regression pass rather than assuming Phases 1-5 already covered them.

Why this phase exists

The brief lists four specific historical bugs by name. Because Phase 1 centralized the logic these bugs stem from, this phase should mostly *confirm* correctness — but it must be done explicitly and deliberately, not skipped on the assumption that “it probably works now.”

What to verify

1. **Incorrect stock totals** — run a mixed sequence of stock-in, stock-out, and transfer operations on the same product/warehouse and confirm the running total matches manual arithmetic at every step.
2. **Negative inventory** — attempt to force negative stock by calling the stock-out and transfer APIs directly (not through the UI) with quantities exceeding available stock, and confirm the request is rejected and stock is unchanged.
3. **Broken transfers** — confirm that every successful transfer updates **both** the source and destination warehouse stock, not just one.
4. **Incorrect low-stock status** — specifically test a product whose stock is *exactly equal* to its reorder threshold, and confirm it is correctly flagged as low stock (using `<=`, never `<`).

Exit criteria

All four problems are explicitly re-tested and confirmed fixed, with the specific inputs used for each noted for the final report.

| Phase 7 — Stretch Feature: Low-Stock Summary

Goal: Optional bonus panel showing, per warehouse, how many products currently require replenishment.

Why this phase exists

This is explicitly marked as bonus and must not delay the core tasks — it's included here for completeness so it isn't forgotten if time permits, but it should only be attempted after Phases 1-6 are fully working.

What to do

1. Only begin this phase once Phases 1-6 are complete and verified.
2. Compute, per warehouse, the count of products where `isLowStock(stock, threshold)` is true — reusing the exact same helper from Phase 1, not a reimplementation.
3. Ensure the panel recomputes when inventory changes, using the same data-fetching/refresh pattern already established for the inventory page.

Exit criteria

The summary panel shows accurate per-warehouse counts and updates after any stock-affecting operation, without introducing a second definition of “low stock.”

| Phase 8 — Final Verification & Report

Goal: Confirm the entire application meets every acceptance criterion end-to-end, and produce the deliverable report.

Why this phase exists

Individual phase testing can miss interaction effects between features. This final pass treats the app as a whole system and produces the audit trail requested in the original brief.

What to do

1. Re-run the **complete** edge case matrix from the original spec (§23) against the actual running application — not just by re-reading the code.
2. Specifically attempt to bypass server-side validation by calling API routes directly with malformed or malicious payloads (bad IDs, negative numbers, non-numeric strings, missing fields) and confirm every case is rejected cleanly with no partial mutation.
3. Re-run the `TODO` search from Phase 0 and confirm each item is either resolved or explicitly and intentionally left with a documented reason.
4. Compile the final report containing:
 - **Files changed** — a list of every modified or newly created file.
 - **Features implemented** — mapped to the task numbers in the original spec (Inventory, Stock In, Stock Out, Transfer, History, Low-Stock Summary).
 - **Bugs fixed** — explicitly mapped to the four numbered problems from Phase 6.
 - **Tests performed** — mapped to the edge case matrices from Phases 3, 4, 5, and the full §23 matrix.
 - **Known limitations** — anything intentionally left out of scope (e.g. real authentication, persistence beyond in-memory data).

Exit criteria

Every checkbox in the original acceptance criteria list is verifiably true, and the report is complete enough that another developer could understand exactly what changed and why without re-reading the whole diff.

| Ordering Rationale (Why the Phases Are Sequenced This Way)

- **Phase 0 before everything:** you cannot safely modify code you don't understand, and the brief explicitly forbids

restructuring or rebuilding — discovery is what makes “focused changes” possible instead of guesswork.

- **Phase 1 before Phases 2-4:** centralizing `applyStockChange` and `isLowStock` up front is what prevents the exact bugs (incorrect totals, `<` vs `<=`, broken transfers) that the brief warns about. Building this logic separately inside each feature is how those bugs happen in the first place.
- **Phase 3 before Phase 4:** Transfer is implemented as two calls to the same stock-adjustment helper used by Stock In/Out. That helper needs to already be correct and tested on the simpler single-warehouse case before it's trusted for the more complex, atomicity-sensitive two-warehouse case.
- **Phase 5 after Phases 3-4:** History is primarily a read/filter/sort layer over transaction records — it can only be properly tested once real transactions are actually being generated by working Stock In/Out/Transfer operations.
- **Phase 6 as a dedicated pass, not folded into 3-4:** even though centralized logic should prevent these bugs, the brief calls them out by name, so they get an explicit, separate verification step rather than being assumed fixed.
- **Phase 7 strictly after Phase 6:** it's optional and must never come at the cost of core correctness, per the brief's explicit priority ordering (functional correctness > data consistency > validation > transaction integrity > filtering/sorting > bonus features > visual polish).
- **Phase 8 last:** a whole-system regression and audit pass catches interaction bugs between features that phase-isolated testing can miss, and produces the deliverable the brief asks for.